11. ROS 2 Parameter Service Case

1. Introduction to Parameters

Similar to global variables in C++ programming, parameters make it convenient to share certain data among multiple programs. Parameters are the global dictionary in the ROS robot system, and data can be shared among multiple running nodes.

In the ROS system, parameters exist in the form of a global dictionary. What is a dictionary? Just like a real dictionary, it consists of a name and a value, also called a key and a value, which together form a key-value pair. Alternatively, we can understand it as being like a parameter in programming: there is a parameter name, followed by an equal sign, and then the parameter value. When using it, you just access this parameter name.

Parameters have very rich characteristics. For example, if a node shares a parameter, other nodes can all access it. If a node modifies the parameter, other nodes also have a way to know immediately, thereby obtaining the latest value.

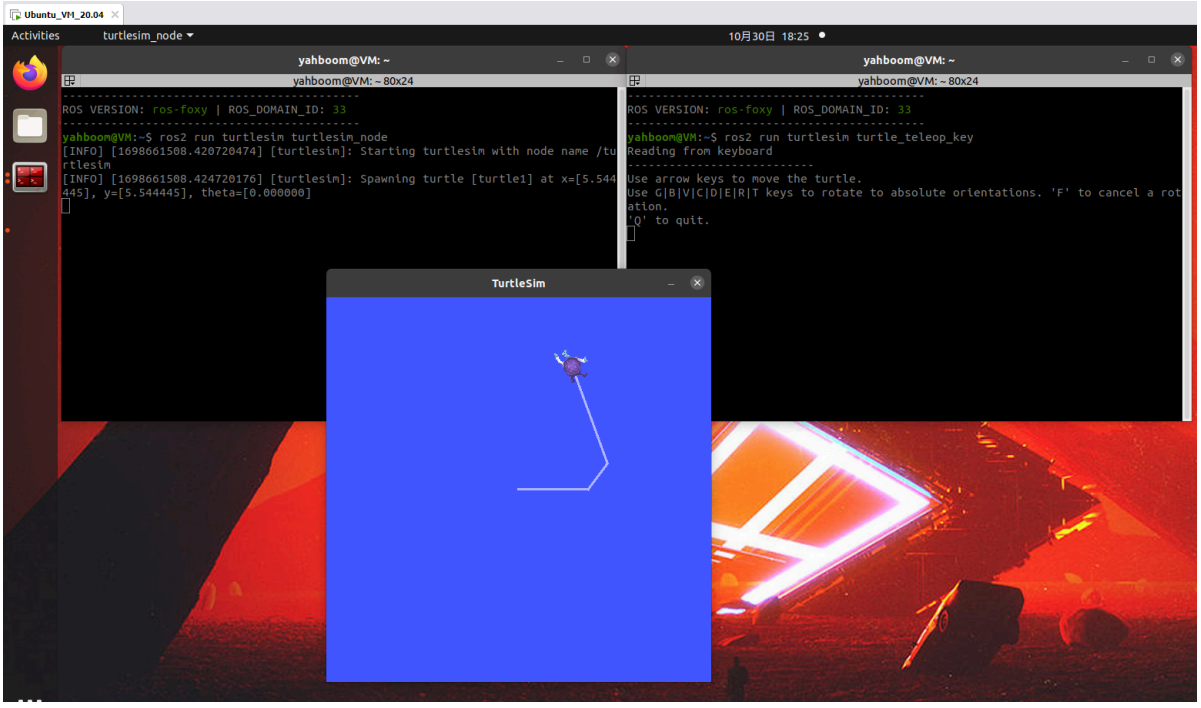
2. Parameters in the Turtlesim Example

In the turtlesim example, the simulator also provides quite a few parameters. Let's use this example to become familiar with the meaning of parameters and the usage of the command line.

Since a GUI needs to be displayed here, for ease of operation, the following case is demonstrated in the virtual machine provided with the tutorial.

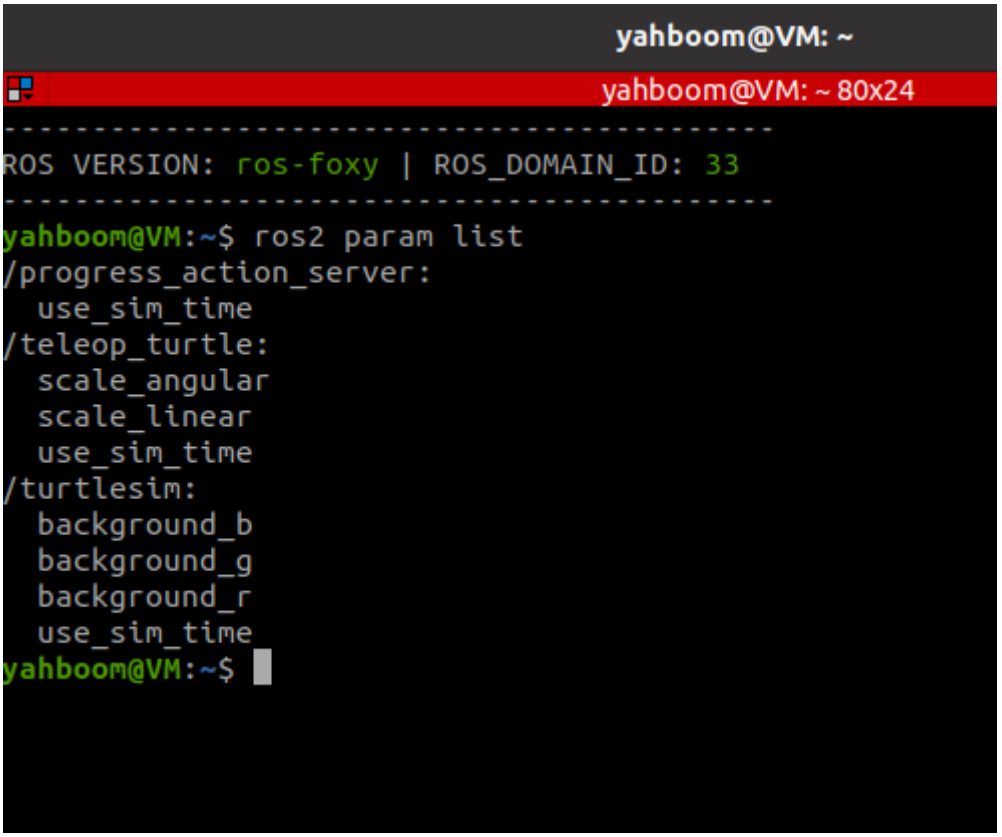
1. Start two terminals and run the turtlesim simulator and the keyboard control node respectively:

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```



2. Start a terminal in the virtual machine and use the following command to view the parameter list

```
ros2 param list
```



3. Parameter query and modification

If you want to query or modify the value of a parameter, you can follow the param command with the get or set subcommand:

```
ros2 param describe turtlesim background_b   # View the description information
of a parameter
ros2 param get turtlesim background_b        # Query the value of a parameter
ros2 param set turtlesim background_b 10     # Modify the value of a parameter
```

4. Saving and loading parameter files

Querying/modifying parameters one by one is too troublesome. Why not try a parameter file? Parameter files in ROS use the yaml format. You can follow the param command with the dump subcommand to save all parameters of a node to a file, or use the load command to load all the content of a parameter file at once:

```
ros2 param dump turtlesim >> turtlesim.yaml  # Save the parameters of a node to a
parameter file
ros2 param load turtlesim turtlesim.yaml     # Load all parameters in a file at
once
```

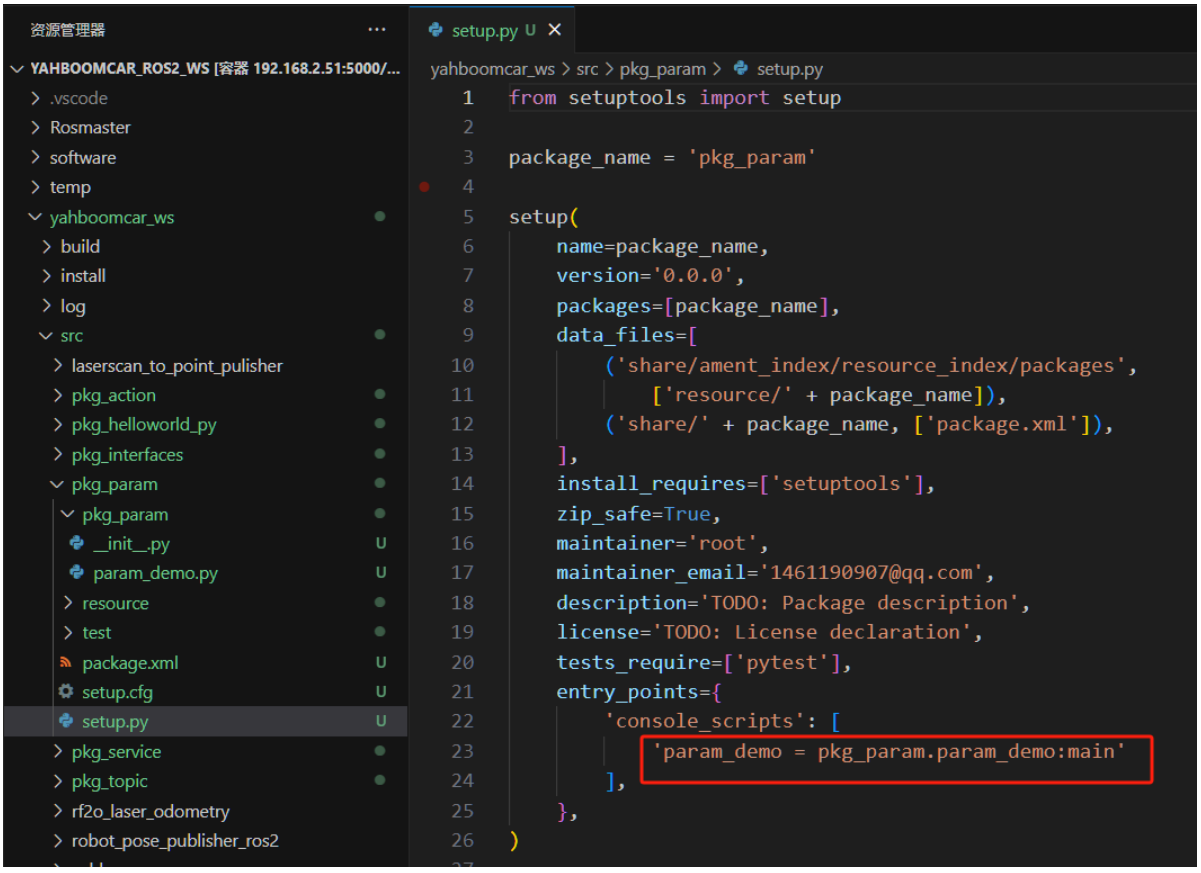
3. Parameter Case

3.1. Creating a New Package

- Create a new package in the src directory of the workspace

```
ros2 pkg create pkg_param --build-type ament_python --dependencies rclpy --node-name param_demo
```

After executing the above command, the pkg_param package will be created, a param_demo node will also be created, and the relevant configuration files will already be configured.



3.2. Code Implementation

Next, edit 【param_demo.py】 to implement the publisher functionality, and add the following code:

```
import rclpy                                    # ROS2 Python interface library
from rclpy.node import Node                     # ROS2 node class

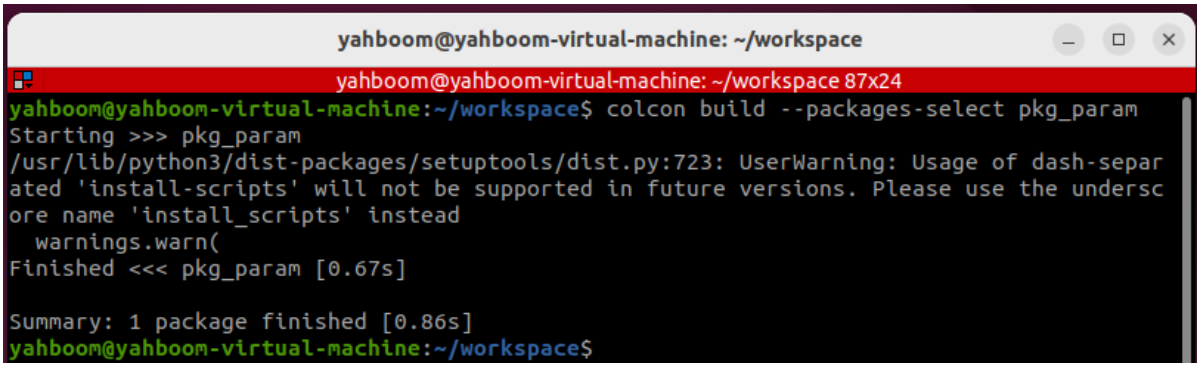
class ParameterNode(Node):
    def __init__(self, name):
        super().__init__(name)                                  # ROS2 node parent class initialization
        self.timer = self.create_timer(2, self.timer_callback)    # Create a timer (period in seconds, the callback function executed periodically)
        self.declare_parameter('robot_name', 'muto')            # Create a parameter and set the default value of the parameter
```

```
    def timer_callback(self):                                   # The callback
function executed periodically by the timer
        robot_name_param =
self.get_parameter('robot_name').get_parameter_value().string_value   # Read the
value of the parameter from the ROS2 system
        self.get_logger().info('Hello %s!' % robot_name_param)     # Output log
information, printing the read parameter value

def main(args=None):                                        # The main entry function of
the ROS2 node
    rclpy.init(args=args)                                   # ROS2 Python interface
initialization
    node = ParameterNode("param_declare")                   # Create a ROS2 node object
and initialize it
    rclpy.spin(node)                                        # Loop and wait for ROS2 to
exit
    node.destroy_node()                                     # Destroy the node object
    rclpy.shutdown()                                        # Shut down the ROS2 Python
interface
```

3.3. Compiling the Package

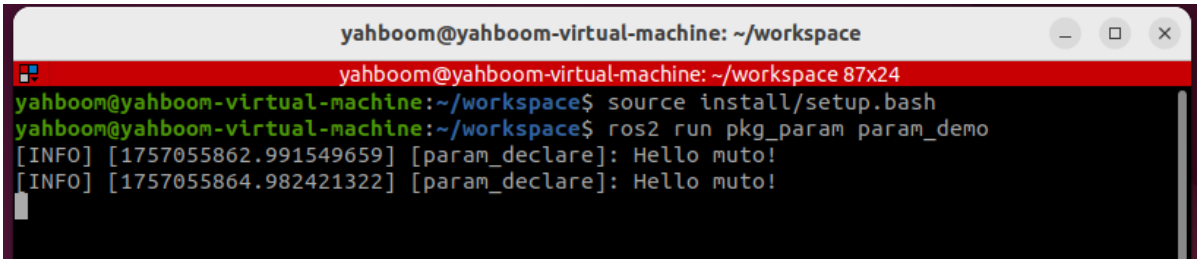
```
colcon build --packages-select pkg_param
```



3.4. Running the Program

- Refresh the environment variables first, and then run the node

```
ros2 run pkg_param param_demo
```



Open another terminal and set robot_name to robot:

```
ros2 param set param_declare robot_name robot
```

In the terminal, you can see the log information printed in a loop, in which "muto" is a parameter value we set by default, and the parameter name is "robot_name". After modifying this parameter through the command line, you can see that the terminal changes accordingly.

